

System Verification and Validation Plan for CEWS System

Lesley Wheat

April 17, 2023

1 Revision History

Date	Version	Notes
2023-02-10	0.0	Creation
2023-02-14	1.0	Version 1.0
2023-04-16	2.0	Revision based on feedback

Contents

1	Revision History	i
2	Symbols, Abbreviations and Acronyms	iv
3	General Information	1
3.1	Summary	1
3.2	Objectives	1
3.3	Relevant Documentation	1
4	Plan	2
4.1	Verification and Validation Team	2
4.2	SRS Verification Plan	2
4.3	Design Verification Plan	2
4.4	Verification and Validation Plan Verification Plan	2
4.5	Implementation Verification Plan	2
4.6	Automated Testing and Verification Tools	3
4.7	Software Validation Plan	3
5	System Test Description	3
5.1	Tests for Functional Requirements and Nonfunctional Requirements	3
5.1.1	Area of Testing 1: Input Testing - FR1, NFR2	3
5.1.2	Area of Testing 2: Calculation Result - FR2, NFR1	6
5.1.3	Area of Testing 3: Graph generation - FR2	7
5.1.4	Area of Testing 4: Output Verification - FR2	9
5.1.5	Area of Testing 5: Reproducibility - FR3	9
5.2	Traceability Between Test Cases and Requirements	11
5.3	Traceability Between Test Cases and Modules	12
	References	13
6	Appendix	14
6.1	Symbolic Parameters	14

List of Tables

1	CEWS Result on Benchmark Datasets from Zighed et al. (2005) . .	7
2	Which test cases are supporting which requirements	11
3	Which test cases are supporting which modules	12
4	Table of SYMBOLIC_CONSTANTS	14

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test
CEWS	Cut Edge Weight Statistic
J_N	Normalized Multiclass Cut Edge Weight Measurement
D	Input Data Matrix
C	Input Class Matrix
FR	Functional Requirement
NFR	Non Functional Requirement

This document outlines the verification plan for the CEWS System.

3 General Information

3.1 Summary

This document contains the verification plan for CEWS System, a program for calculating the CEWS measurement for a dataset as shown in [Zighed et al. \(2005\)](#). CEWS System takes a set of data points D and a list of class labels C as inputs and computes the Normalized Multiclass Cut Edge Weight Measurement J_N .

3.2 Objectives

The objective of this document is to build confidence in the software correctness. This document provides the test plan used to verify the final product. Therefore, it should aim to encompass all verification components required for testing. No requirements have been excluded. Software validation is considered out of scope at this time.

3.3 Relevant Documentation

Relevant documentation:

- System Requirements Specification (SRS)
 - [GitHub Link](#)
 - Describes the requirements, explains notation and theory.
- System Requirements Specification Checklist
 - [GitHub Link](#)
 - Checklist for reviewing the SRS.
- Module Guide (MG)
 - [GitHub Link](#)
 - Describes the modules and how they connect back to the requirements.
- Module Interface Specification (MIS)
 - [GitHub Link](#)
 - Contains more detailed information about the modules.

- Module Interface Specification Checklist
 - [GitHub Link](#)
 - Checklist for reviewing the MIS.

4 Plan

4.1 Verification and Validation Team

The verification and validation team includes the domain expert Joachim de Fourestier, the verification and validation reviewer Mina Mahdipour, and Dr. Spencer Smith.

4.2 SRS Verification Plan

The SRS will be reviewed by the SRS review team, consisting of the domain expert Joachim de Fourestier and the SRS reviewer, Sam Joseph Crawford, and Dr. Spencer Smith. The SRS review will be done with the [SRS checklist](#).

4.3 Design Verification Plan

The design document MIS will be reviewed by the MIS review team, consisting of the domain expert Joachim de Fourestier and the MIS reviewer, Deesha Vipulkumar Patel, and Dr. Spencer Smith. The MIS review will be done with the [MIS checklist](#).

4.4 Verification and Validation Plan Verification Plan

This document will be reviewed by all the VnV team members in Section [4.1](#).

4.5 Implementation Verification Plan

The software will be implemented in Python. Individual tests are outlined in Section [5](#). The tests will be automated using PyTest. The following benchmark datasets from the UCI Machine Learning Repository will be used: Breast Cancer, Glass Ident, Haberman, Image seg, Ionosphere, Iris (Bezdek), Iris plants, Wine recognition and Yeast.

To test the output of the software, results from the paper [Zighed et al. \(2005\)](#) will be used as a pseudo-oracle. This is used by Test [10](#) with the benchmark datasets. To test the output of the graph module, the [relativeNeighborhoodGraph Cavanagh](#) python package will be used as a pseudo-oracle. This is used by Test [11](#) with the benchmark datasets.

4.6 Automated Testing and Verification Tools

The VnV process will use PyTest for testing.

4.7 Software Validation Plan

Software Validation is out of scope.

5 System Test Description

5.1 Tests for Functional Requirements and Nonfunctional Requirements

5.1.1 Area of Testing 1: Input Testing - FR1, NFR2

This area of testing will address inputs to the program.

1. T1 - Normal Input

Control: Automated

Initial State: Pending Input

Input: Meets all constraints from SRS.

- $D = [[1, 0], [0, 1], [0, 1]]$
- $C = [0, 1, 2]$

Output: No error generated, program outputs a result.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module, Calculation Module, Graph Module, Output Module

How test will be performed: PyTest on Windows 11 operating system

2. T2 - Data Mismatch: More Labels

Control: Automated

Initial State: Pending Input

Input: C has more samples than D .

- $D = [[1, 0], [0, 1], [0, 1]]$
- $C = [0, 1, 2, 3]$

Output: Generate ValueError.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module

How test will be performed: PyTest on Windows 11 operating system

3. T3 - **Data Mismatch: More Data**

Control: Automated

Initial State: Pending Input

Input: D has more samples than C .

- $D = [[1, 0], [0, 1], [0, 1]]$
- $C = [0, 1]$

Output: Generate ValueError.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module

How test will be performed: PyTest on Windows 11 operating system

4. T4 - **Single Class**

Control: Automated

Initial State:

Input: C contains only one class.

- $D = [[1, 0], [0, 1], [0, 1]]$
- $C = [0, 0, 0]$

Output: Generate ValueError.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module

How test will be performed: PyTest on Windows 11 operating system

5. T5 - **Wrong Number of Features: Data**

Control: Automated

Initial State:

Input: D has a different number of features.

- $D = [[1, 0, 0], [0, 1], [0, 1]]$

- $C = [0, 1]$

Output: Generate TypeError.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module

How test will be performed: PyTest on Windows 11 operating system

6. T6 - **Wrong Size: Labels**

Control: Automated

Initial State:

Input: C is not a matrix.

- $D = [[1, 0], [0, 1], [0, 1]]$
- $C = [0, 1, [1, 1]]$

Output: Generate TypeError.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module

How test will be performed: PyTest on Windows 11 operating system

7. T7 - **Wrong Value Type in Data Matrix**

Control: Automated

Initial State: Pending Input

Input: D contains a string.

- $D = [[1, 0], [0, ?], [0, 1]]$
- $C = [0, 0, 1]$

Output: Generate TypeError.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module

How test will be performed: PyTest on Windows 11 operating system

8. T8 - **Wrong Value Type in Label Matrix**

Control: Automated

Initial State: Pending Input

Input: C contains a string.

- $D = [[1, 0], [0, 0], [0, 1]]$
- $C = [0, ?, 1]$

Output: Generate TypeError.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module

How test will be performed: PyTest on Windows 11 operating system

9. T9 - Labels Not a Vector

Control: Automated

Initial State: Pending Input

Input: C is not of size $n \times 1$.

- $D = [[1, 0], [0, 0], [0, 1]]$
- $C = [[1, 0], [0, 0], [0, 1]]$

Output: Generate TypeError.

Test Case Derivation: FR1, NFR2

Modules: Control Module, Input Module

How test will be performed: PyTest on Windows 11 operating system

5.1.2 Area of Testing 2: Calculation Result - FR2, NFR1

This area of testing addresses the result of the program. The absolute error should be less than `ACCURACY_ERR_ABS`. The absolute error is used because the error should not vary based on the size of the result (the scale of the values is limited).

1. T10 - Output on UCI Dataset Tests

Control: Automated

Initial State: Pending Input

Input: UCI Machine Learning Repository datasets: Breast Cancer, Haberman, Glass Identification, Image seg, Ionosphere, Iris (Bezdek), Iris, Wine recognition and Yeast.

Output: Normalized CEWS Measurement J_N within `ACCURACY_ERR_ABS`.

Test Case Derivation: FR2, NFR1

Modules: Control Module, Input Module, Calculation Module, Graph Module, Output Module

How test will be performed: PyTest

Test steps:

- (a) Format a dataset
- (b) Input the reformatted dataset to the system
- (c) Check the output value against the results in Table 1.

Dataset	Expected Result
Breast Cancer	0.008
Glass Identification	0.356
Haberman	0.331
Image segments	0.224
Ionosphere	0.137
Iris (Bezdek)	0.090
Iris Plants	0.087
Wine recognition	0.093
Yeast	0.524

Table 1: CEWS Result on Benchmark Datasets from [Zighed et al. \(2005\)](#)

5.1.3 Area of Testing 3: Graph generation - FR2

This area of testing addresses the graph generation of the program and output verification of the program.

1. T11 - Graph Module and relativeNeighborhoodGraph Test on UCI Dataset Tests

Control: Automated

Initial State: Pending Input

Input: UCI Machine Learning Repository datasets: Breast Cancer, Haberman, Glass Identification, Image seg, Ionosphere, Iris (Bezdek), Iris, Wine recognition and Yeast.

Output: The Graph Module connection matrix.

Test Case Derivation: FR2

Modules: Graph Module

How test will be performed: PyTest

Test steps:

- (a) Format a dataset
- (b) Input the reformatted dataset to the system
- (c) Record the connection matrix, the output of the graph module.
- (d) Check the connection matrix matches the connection matrix generated by the relativeNeighborhoodGraph Python library [Cavanagh](#).

2. T12 - Graph Generation: Square

Control: Automated

Initial State: Pending Output

Input: A dataset of points that form a square.

- $D = [[0, 0], [1, 0], [0, 1], [1, 1]]$

Output: A connection matrix that has edges only on the edge of the square.

- $\text{output} = [[0, 1, 1, 0], [1, 0, 0, 1], [1, 0, 0, 1], [0, 1, 1, 0]]$

Test Case Derivation: FR2

Modules: Graph Module

How test will be performed: PyTest

3. T13 - Graph Generation: Line

Control: Automated

Initial State: Pending Output

Input: A dataset of points that form a line.

- $D = [[0, 0], [1, 1], [2, 2], [3, 3]]$

Output: A connection matrix that has edges only connecting the closest points.

- $\text{output} = [[0, 1, 0, 0], [1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0]]$

Test Case Derivation: FR2

Modules: Graph Module

How test will be performed: PyTest

5.1.4 Area of Testing 4: Output Verification - FR2

This area of testing addresses the graph generation of the program and output verification of the program.

1. T14 - Output Out of Bounds: Too Large

Control: Automated

Initial State: Pending Output

Input: $J_N > J_{\max}$

- $J_N = 1.1$

Output: Generate ValueError.

Test Case Derivation: FR2

Modules: Output Module

How test will be performed: PyTest

2. T15 - Output out of Bounds: Too Small

Control: Automated

Initial State: Pending Output

Input: $J_N < J_{\min}$

- $J_N = -0.1$

Output: Generate ValueError.

Test Case Derivation: FR2

Modules: Output Module

How test will be performed: PyTest

5.1.5 Area of Testing 5: Reproducibility - FR3

This area of testing addresses the reproducibility of the program.

1. T16 - Reproducibility on UCI Dataset Tests

Control: Automated

Initial State: Pending Input

Input: UCI Machine Learning Repository datasets: Breast Cancer, Haberman, Glass Identification, Image seg, Ionosphere, Iris (Bezdek), Iris, Wine recognition and Yeast.

Output: Normalized CEWS Measurement J_N .

Test Case Derivation: FR3

Modules: Control Module, Input Module, Calculation Module, Graph Module, Output Module

How test will be performed: PyTest

Test steps:

- (a) Format a dataset
- (b) Input the reformatted dataset to the system
- (c) Record the value J_N
- (d) Repeat SHUFFLE_TIMES: Shuffle the input dataset and recalculated the value of J_N , compare this value against the first value of J_N .

5.2 Traceability Between Test Cases and Requirements

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15	T16
FR1	X	X	X	X	X	X	X	X	X							
FR2										X	X	X	X	X	X	
FR3																X
NFR1										X						
NFR2	X	X	X	X	X	X	X	X	X							

Table 2: Which test cases are supporting which requirements

5.3 Traceability Between Test Cases and Modules

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15	T16
Control Module	X	X	X	X	X	X	X	X	X	X						X
Input Module	X	X	X	X	X	X	X	X	X	X						X
Output Module	X									X				X	X	X
Calculation Module	X									X						X
Graph Module	X									X	X	X	X			X

Table 3: Which test cases are supporting which modules

References

- Tem Cavanagh. Relative neighborhood graph. <https://github.com/temcavanagh/relativeNeighborhoodGraph>. Accessed: 2023-04-02.
- Djamel A. Zighed, Stphane Lallich, and Fabrice Muhlenbach. A statistical approach to class separability. *Applied Stochastic Models in Business and Industry*, 21(2):187–197, March 2005. ISSN 1524-1904, 1526-4025. doi: 10.1002/asmb.532. URL <https://onlinelibrary.wiley.com/doi/10.1002/asmb.532>.

6 Appendix

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

Table 4: Table of SYMBOLIC_CONSTANTS

Var	Value
$F_{\min}$	$-(2 - 2^{-23})2^{127}$
$F_{\max}$	$(2 - 2^{-23})2^{127}$
$W_{\min}$	0
$W_{\max}$	$2^8 - 1$
$J_{\max}$	1
$J_{\min}$	0
ACCURACY_ERR_ABS	0.1
SHUFFLE_TIMES	3